ISHTIRAK

Distributed Neighborhood Capacity Coordination

Final project report | Distributed Systems | 29 October 2026

A finite generator budget, coordinated through a majority-committed ledger.

Three Go coordinators agree on capacity grants. Building gateways aggregate MQTT readings and enforce short-lived budgets in a simulated served-load calculation. PostgreSQL preserves audit and telemetry, and a React dashboard exposes normal operation and controlled failures.

120 A capacity. Three coordinators. Eleven integration phases passed.

The measured run admitted 17 of 40 competing 7 A requests, reserving 119 A. No capacity violations were observed in 708 coordinator samples, nor in a randomized fault check of 1,529 requests. The report distinguishes this evidence from a formal proof and explains the timing assumptions and prototype limits.

Application: http://localhost:8088

Source, reproduction steps, raw measurements, and an editable PowerPoint accompany this report.

1. Problem and outcome

A neighborhood generator has a finite amperage budget, while buildings independently observe changing household demand. If buildings make allocation decisions independently, concurrent requests or retries can reserve more than the generator can supply. A coordinator crash introduces a second problem: a replacement must distinguish capacity that is still authorized from capacity that can safely be reused.

Ishtirak addresses this coordination problem with a replicated capacity ledger and short-lived building leases. The prototype uses Go meter simulators, MQTT/Mosquitto, three Go building gateways, three HashiCorp Raft coordinators, PostgreSQL, and a React dashboard. Its central invariant is:

Total granted integer amperage <= committed generator capacity.

The supplied demonstration uses a 120 A generator and initially 50 simulated meters across Hamra, Mar Mikhael, and Achrafieh. A gateway computes simulated served demand as the smaller of observed demand and its currently valid lease budget. No physical breaker, relay, or electrical installation is controlled.

The implemented application, evaluation scripts, report, and PowerPoint cover the deliverable categories in the supplied project brief. The original brief, proposal, and flow diagram remain unchanged. Assessment of the final grade remains with the instructor.

2. Why this is a distributed system

The system has autonomous processes with separate memory and explicit network communication. Each coordinator has its own persistent Raft log and snapshot volume. There is no shared-memory capacity map between coordinators. MQTT messages connect meter producers to building consumers; HTTP connects gateways to the coordinator service; Raft replicates ordered commands over TCP; SQL stores a recoverable projection; HTTP and WebSocket expose observations to the dashboard.

Component	Partition or responsibility	Communication and state
Meter simulator	Independent logical household meters; 10/50/200 evaluated	MQTT QoS 1; unique reading IDs
Mosquitto	Decouples producers from building subscribers	Topic-based publish/subscribe
Three gateways	One building per process; aggregate its meter readings	MQTT input, HTTP grants/readings; local lease deadlines
Three coordinators	Replicated authority over the shared generator budget	Raft TCP; HTTP/JSON and WebSocket; three separate volumes
PostgreSQL	Durable audit and reading projection	SQL; single database, no grant authority
React + local controller	Operator visibility and bounded fault demonstrations	HTTP/WebSocket; label-checked Docker actions

The brief gives "Database, Application and Logging service" as example components. In Ishtirak the **database** is PostgreSQL, the **application** is the meter simulators, gateways, coordinators and dashboard, and the **logging service** is the replicated audit log: every committed grant or reclamation becomes an event in the Raft state machine's outbox, is projected by the observed leader into an append-only SQL audit table with deterministic event IDs, and is shown and exportable in the dashboard's Activity log. There is no separate logging process; logging is a responsibility of the coordinators and PostgreSQL, and it keeps catching up if PostgreSQL is unavailable.

There are eleven Compose services. Meter identities are multiplexed by one simulator process rather than launching 200 containers. The dashboard and controller are distinct services. Compose runs these processes on one host: this demonstrates distributed protocols and process/network failures, but does not tolerate loss of that host. The local fault controller is demonstration infrastructure, not an additional source of allocation authority.

Communication path

1. Meters publish identified readings to building-specific MQTT topics.

2. A building gateway deduplicates readings, removes stale demand samples, and aggregates demand.

3. The gateway asks a coordinator for a building lease. Followers redirect it to the known leader; redirects are restricted to configured coordinator endpoints.

4. The leader appends a command to Raft and waits for its committed state-machine result.

5. The gateway accepts only a matching, timely reply and applies its safe local deadline.

6. Audit events are projected from committed Raft state into SQL. Reading batches are acknowledged only after durable SQL insertion.

7. The UI polls combined observations and consumes live coordinator WebSocket updates. Its status values are observations, not a linearizable read of the entire distributed system.

The implementation separates telemetry partitioning by building from replication of the single shared capacity budget. Adding a building partition can scale telemetry handling, but every allocation still passes through one logical consensus ledger.

3. Deterministic consistency and concurrency

The ledger starts uninitialized. A replicated init_capacity command establishes capacity once; later nodes replay the same value instead of deriving it from local configuration. Grants fail closed before initialization. Incompatible legacy histories fail closed rather than being reinterpreted as safe capacity state.

Each grant request contains request_id, building_id, session_id, and positive integer amps. The state machine stores a canonical payload hash with the result. Repeating the same request returns the stored result; changing the payload under that ID produces request_id_conflict. A timeout is an unknown outcome, so a gateway retries the original ID and retains its original send time.

Raft serializes commands, and the ledger protects state access with a mutex. Admission computes the incremental reservation and compares it with available capacity. A renewal for the same session adds a tier, a committed grant a gateway may still be using; the reservation is the largest tier and shrinks only

when an older, larger tier has expired (Section 4). A different session cannot replace a currently reserved building lease. Each successful grant version is its unique committed log index.

An induction argument explains the accounting invariant. Initialization creates an empty ledger with positive capacity. A grant increases reservations only if its increment fits the available capacity. Rejected commands and duplicate requests do not increase reservations. Expiry only removes the matching reservation. Consequently, each committed transition preserves the capacity bound. This argument depends on the implementation matching the transition rules; unit tests, review, race checks, and live stress supply evidence for that correspondence.

Snapshot restoration includes initialized capacity, leases, request hashes and outcomes, the audit counter, and the event outbox. Restore validation rejects invalid or over-capacity state. It does not discard idempotency history merely because a log has been compacted.

4. Leases, failures, and timing assumptions

Lease safety is intentionally conservative. The protocol has a fixed 12-second lease duration, enforced at coordinator startup. A gateway may use an accepted grant only until its original first-send monotonic time plus 10 seconds. The two-second margin is subtracted locally. Retries and duplicate replies never move that original send time forward. Delayed, expired, or identity-mismatched replies are rejected.

The coordinator holds a newly observed lease version for 12 monotonic seconds before proposing expiry. A leadership-term change restarts inherited holds. Expiry is a replicated command fenced by the building ID, expected lease version, and proposing Raft term. A stale leader or stale expiry cannot erase a newer version. The ledger itself never consults a wall clock to reclaim capacity.

With a relative monotonic clock-rate error bound rho, the gateway's latest stop must precede the coordinator's earliest expiry: 10/(1-rho) < 12/(1+rho). The documented operational assumption is at most 1% rate error. At that bound, the respective durations are about 10.10 and 11.88 real seconds; the coordinator also starts its hold no earlier than observation of the committed lease. Absolute wall-clock offsets do not decide safety. Host suspend behavior, malicious processes, and physical actuation delays are outside the certification of this simulator.

Event	Designed response	Trade-off
One coordinator fails	Two remaining nodes elect a leader and commit	Temporary interruption during election
Majority is unavailable	No new grant can commit; local service deadlines expire	Availability is sacrificed to prevent conflicting grants
Gateway crashes	Its reservation remains until committed expiry; restart has a fresh session	Reconnection can wait for conservative reclamation
A response is delayed or duplicated	Original send time and result identity remain binding	A valid committed grant may arrive too late to use
PostgreSQL fails	Raft grants continue; audit outbox catches up after recovery	Reads/history are degraded; bounded gateway queues may lose data
Broker or gateway is unavailable	Telemetry becomes missing/stale; current lease rules still apply	No broker or gateway replication

Lease tiers. A gateway keeps every grant it receives with its own deadline and serves up to the largest one still valid, so the ledger must keep reserving that largest amount until the grant can no longer be used. Each lease therefore carries tiers: the committed grants that may still be in use, with strictly decreasing amounts as they get newer. The reservation is the largest tier. A renewal adds a tier (dropping older tiers it dominates, because a newer grant of equal or larger size expires no later), the coordinator holds each tier for its own 12 s after first observing it, and an expiry command removes that tier and every older one. The reservation therefore steps down to the building's current demand by itself while the gateway keeps renewing, and is never reduced while an older, larger grant could still be served under. A grant returns only its own amount, never the reserved maximum, so a gateway cannot extend an old large authorization. docs/LEASE_TIERS.md gives the safety argument and its assumptions: the same bounded clock-rate-error assumption as above, and all three coordinators must be upgraded together because an old replica ignores tier expiries. Replaying an old command log gives the same decisions as before the change, and old snapshots load with one implicit tier. Tiers make reservations shrinkable; the fair-sharing policy below decides who shrinks. The system demonstrates safety under constrained availability and does not promise optimal utilization.

The tiers were added because of a failure observed live. Originally a renewal never shrank a reservation: after a jump from a quiet network to 200 meters, one building took about 100 A first-come and kept it for as long as its gateway renewed, while the other buildings had 0 A (holds of 107 to 111 A against about 26 A of demand, until the gateway restarted). The invariant held, but utilization and fairness did not. With tiers the same scenario recovers by itself: in evaluation run c6d1e46130fc one building reserved 99 A during the spike while another had no reservation, and 17.6 s after demand returned to 50 meters every building was again within 1.5 x its demand + 3 A (24, 22 and 26 A reserved). The decay takes about one hold (12 s) after demand falls.

Tiers alone were still first-come: while two buildings both demanded more than the free capacity the first kept it. That is the problem the next section addresses; scripts/reset_demo.py (restart the gateways) remains available for a clean demonstration state.

Fair sharing. Under overload the system now divides the generator by max-min fairness. The admission policy is committed state: first_come (the previous rule, still selectable) or fair_share (the default for a

new cluster). It is a set_policy log entry, not node-local configuration, so every replica and every replay switches at the same log index; a coordinator proposes its configured policy only while the ledger has none. Every processed request records the building's *want* (its latest requested amount, clamped to the capacity, bounded to 64 buildings and 256 for leaseholders, and dropped by a term-fenced expire_want after one 12 s hold without a refresh). A grant authorizes the lesser of the want and the building's max-min share, and growth is still limited to the free capacity at that command, so granted <= capacity holds whatever the shares or wants are: wants and shares only affect fairness. A building above its share is not revoked. Its renewals are capped, its older, larger tier is no longer refreshed and lapses on its own hold, and the freed amps become available to the starved buildings, which receive a partial grant (granted_partial) instead of a refusal. At 120 A, wants of 102/99/98 A settle at 40/40/40 A and 10/100/100 A at 10/55/55 A. The reservations settle after about one hold (12 s) plus a gateway round trip; a leader change restarts the hold of inherited tiers. An old coordinator would ignore the new commands and diverge, so a background check keeps fair_share off, and refuses POST /api/policy with 409, while any coordinator is unreachable or does not report the fair_share status key; the coordinators must be upgraded together. The endpoint also requires an admin header that only the control service holds (defense in depth with a public demo token, not authentication). docs/FAIR_SHARING.md gives the rules, the safety argument and the review findings.

Limits of fair sharing: every building is equal (no priority classes), building ids are not authenticated so a client that can reach a coordinator can invent buildings (bounded, and it never evicts a building that holds a lease), a silent building's want caps the others for up to one hold, a request above the whole generator is clamped to the capacity, and a request history grows without bound under sustained overload.

5. Persistence and error handling

Three named volumes retain the coordinators' logs and snapshots. PostgreSQL has its own volume. Deterministic audit event IDs and SQL primary keys make projection retries idempotent. The observed leader exports committed audit events; the FSM retains the outbox for catch-up. Reading IDs similarly prevent duplicate SQL rows.

MQTT QoS 1 can deliver duplicates. Gateways deduplicate readings and use bounded queues; pending and dropped counts are visible. Failed SQL flushes retain their batch until a later retry. Readings during gateway downtime may be lost, and a gateway crash loses its in-memory buffer. These limitations match the proposal's unreplicated gateway and single-database scope.

HTTP handlers validate and bound input sizes. The local control service accepts only named actions for this project's containers and checks their Compose labels before Docker operations. The control service rejects cross-origin browser requests, and the dashboard's proxy accepts only localhost and 127.0.0.1 Host values (HTTP 421 otherwise, which closes DNS-rebinding hostnames) and refuses browser-originated writes to the coordinators (HTTP 403; local scripts send no Origin header and are unaffected). There is still no authentication on the coordinator or control APIs. The dashboard is bound to 127.0.0.1:8088. The controller's Docker socket and the public demo database password make this a trusted local demonstration, not a public deployment configuration.

Raft request history and the audit outbox are retained without an unbounded-production retention policy. This improves prototype replay clarity but creates memory/snapshot growth. Resetting Raft history while preserving old SQL event IDs can invalidate projection correspondence; only an intentional, coordinated reset of the complete stopped demo is appropriate.

6. Measured evaluation

The live run on 29 October 2026 is stored in artifacts/evaluation.json, run ID c6d1e46130fc, on the code with lease tiers. It ran from 12:32:09 to 12:39:08 UTC against the actual Docker stack. The runner uses Python's standard library, bounded timeouts, and a finally restoration path. All twelve phases passed, all controlled services were restored, and the meter configuration returned to 50 meters with demand factor 1. Latency percentiles use linear interpolation between order statistics. Gateway-crash, leader-failure, load and end-to-end measurements are repeated (3, 3, 5 and 5 trials); the tables give the mean, the sample standard deviation (sd) and the range, and evaluation.json keeps every trial. An earlier run on the code before the lease-tier change (f85446506835, eleven phases) is kept as artifacts/evaluation-before-tiers.json for the comparison below.

Capacity and fault results

Scenario	Measured result
Real MQTT and SQL readiness	710,263 durable readings already present at readiness
Gateway crash and reclamation (3 trials)	Lease reclaimed 14.38, 13.58 and 12.89 s after the crash (mean 13.62, sd 0.74); each restart used a fresh session. The crashed building was whichever one held a lease
Leader failure (3 trials)	The leader was stopped three times in turn (coordinator2, then 1, then 2); a replacement accepted a grant after 4.05, 6.39 and 4.05 s (mean 4.83, sd 1.35)
Majority loss	Three new requests returned HTTP 503; simulated service was 0 A after 13 s
PostgreSQL outage	Grant accepted while SQL was down; its exact audit request appeared once after recovery; audit rows rose from 19,543 to 19,594; 1,184 additional durable readings; 50 readings pending during the outage
Concurrent capacity stress	Forty requests of 7 A: 17 accepted, 23 rejected, 119 A committed under 120 A
Duplicate and conflict	Duplicate preserved the lease; changed payload under the same ID rejected

The leader-failure measurement includes routing and a successful grant, not just election time. Gateway reclamation is an observation made through the orchestration path, not an exact internal timer measurement. A 503 during majority loss correctly expresses unavailable/unknown commit outcome, not a definitive rejection that may safely be retried under a new ID.

The sampler collected 323 combined overviews and 947 coordinator snapshots, with zero transport errors, a maximum observed grant of 120 A (the limit, not above it), and no sampled invariant violations. It also checked gateway served demand against local valid budgets. Polling cannot prove that no transient existed between observations; deterministic state-machine checks and timing design supply complementary evidence.

Randomized fault check

scripts/chaos_check.py is a small history checker in the spirit of Jepsen. With a fixed seed it builds a deterministic fault schedule, runs eight concurrent workers that send randomized grants, duplicate replays and one changed-payload replay to random coordinators while the schedule stops and restarts services, samples the capacity ledger about once a second, then heals everything and checks the recorded history. Run 1a5da564 (seed 1, 90 s, 12:30:37 to 12:32:08 UTC, on the code with lease tiers) stopped and restarted coordinator2, the Hamra gateway twice and the Mar Mikhael gateway, and included a majority-loss window in which coordinator3 and coordinator1 were down together for about 15 s.

Check	Result
Requests sent	1,559: 427 accepted, 1,055 rejected, 77 unknown outcomes
Rejections	1,054 capacity_exhausted; 1 request_id_conflict (the deliberate changed-payload replay)
Unknown outcomes	HTTP 503 or timeout, counted separately because they may have committed
Capacity invariant	134 samples; maximum observed grant 120 A (the limit, not above it); 0 violations
Unique versions	No committed version was returned for two different request IDs
Recovery	All seven faultable services restarted; the three coordinators reported the same applied index (40,791)

The synthetic chaos-* buildings share the 120 A budget with the real gateways, which is why capacity rejections dominate. Like the evaluation sampler, this is observational evidence from one seed, not a proof; the unit tests show the checker flags an over-capacity history and duplicate versions.

Performance and scaling

Each load settled for 10 seconds, then ran five trials of 200 allocation requests with four concurrent HTTP clients, using a new synthetic benchmark building and session per trial and unique request IDs, alongside the real building gateways. All 200 decisions in every trial were accepted; the separate capacity stress tested denials. No benchmark had a transport/HTTP error. Values are mean ± sd over the five trials (range in brackets).

Active meters	p50 ms	p95 ms	Decisions/s
10	86.6 ± 6.7 (80.7-94.6)	133.1 ± 10.1 (120.1-143.3)	43.9 ± 2.6
50	99.8 ± 18.9 (87.1-133.1)	167.0 ± 40.1 (137.4-235.8)	38.0 ± 5.5
200	107.5 ± 16.7 (91.8-130.0)	156.7 ± 23.8 (126.0-182.1)	36.4 ± 5.4

Observed gateway MQTT readings per second in the median trial's window were 9.6, 48.3 and 191.6 for 10, 50 and 200 meters, and SQL insert deltas were 57, 284 and 1,133 readings. Batching and observation boundaries explain why these differ from the nominal rates.

These are HTTP allocation-decision latencies, including local proxy/network and commit time. The decision benchmark does not exercise meter volume, so its flat latency across meter counts says little about meter scalability by itself; the meter and reading volume is shown by the readings per second above.

Concurrency sweep. At 50 meters the same 200-decision method was run with 1, 4, 16 and 64 concurrent clients:

Clients	p50 ms	p95 ms	Decisions/s
1	35.9	53.7	26.0
4	86.9	140.6	42.7
16	308.1	485.1	47.5
64	336.3	603.2	47.3

There were no transport errors. Throughput rises with load and then plateaus near 47 decisions/s while latency grows, which is the shape expected when requests queue behind one serialized resource. Every grant is ordered by the single Raft leader, so this is consistent with the commit path being the limit, but the cause was not profiled: the dashboard proxy and the Python client are also in the path. The plateau is therefore a measured ceiling for this setup, not an identified bottleneck.

End-to-end telemetry latency. Stepping the demand multiplier from 0.5 to 1.0 at 50 meters, the change became visible in the gateways' observed demand 0.39 to 0.89 s after the request was sent (mean 0.63 s, sd 0.19 s, five samples; polling every 0.1 s, largest poll gap 193 ms). This covers the control request, meter publish, MQTT, gateway aggregation and dashboard refresh, and is an upper bound at the polling resolution. Served demand moved at the same instant in each sample because a gateway computes it from the same snapshot (the smaller of demand and its lease budget); when a lease budget is the limit, served demand stays flat while demand rises.

Allocation recovery after a demand spike. The twelfth phase runs at 10 meters for 20 s (reservations decay to a few amps), jumps to 200 meters for 10 s, then returns to 50 meters. During the spike one building reserved 99 A and one building had no reservation (first-come); 17.6 s after the return every building was again within budget (reserved 24, 22 and 26 A). Before lease tiers the same sequence left the first building's large reservation in place for as long as its gateway renewed.

Comparison with the run before lease tiers. The earlier run (artifacts/evaluation-before-tiers.json) measured a p50 of 78.3 to 79.5 ms and 48.6 to 50.0 decisions/s at the three meter counts with four clients, against 86.6 to 107.5 ms and 36.4 to 43.9 decisions/s in this run; the one-client p50 was 38.7 ms and then 35.9 ms. This is not a controlled comparison: the second run was measured while PostgreSQL was using about 250% CPU and some thirty unrelated containers were running on the same laptop, and its trial spread is larger (sd up to 19 ms). The unchanged one-client latency suggests the ledger change is not the cause, but that was not isolated, so the report claims no measured cost or benefit of tiers for decision speed. Failover (4.82 s, then 4.83 s) and reclamation (12.3 s, then 13.6 s) are similar.

One host, one run per configuration, short windows, concurrent local activity, and no network-delay injection limit these conclusions, and the trials are repeats on one stack rather than independent samples. Further evaluation should use dedicated hosts, profiling of the commit path, longer soak tests,

and deliberate packet loss and partition schedules. There is no measured claim of unlimited scale.

Software and browser checks

The integrated Go code passed go test -race ./... and go vet ./... in golang:1.25. There are 102 named test functions covering fair sharing (the policy switch as a committed command, the bounded want set, a max-min oracle, an independent model written from the spec, a safety simulation with policy flips and restarts, and the coordinator's peer-version gate and admin token), the lease tiers (a seeded property test of 6,000 random commands checking every invariant and deterministic replay, an oracle model without pruning, agreement with the old rules on old-style logs, snapshot and legacy-snapshot restore, and a simulation with delayed and reordered commits and leader changes), ledger concurrency, duplicate/conflict handling, version/term expiry fences, snapshot restoration, capacity initialization, fixed TTL, inherited deadlines, late replies, gateway restart identities, bounded reading queues, redirects, simulator configuration, and local control restrictions. The Python tooling has 88 unit tests (the fault checker, the statistics helpers and the evaluation phases against an in-memory fake stack) and the dashboard has 165 Vitest tests in seven files (scenario logic, fairness arithmetic, gauge geometry, the peak tracker, the capacity readouts, the console's presentation logic, ledger and cluster-detail logic, and the demo simulator's rules).

Fair sharing on real processes. The measured Docker evaluation above (run c6d1e46130fc) ran the lease-tier code, before fair sharing existed; the evaluation script now has a fair-sharing phase. A live run of the full evaluation on the Docker stack with the fair-sharing build (artifacts/evaluation-fair-share-partial.json) passed its first four phases (readiness with real MQTT, three gateway-crash trials, three leader-failure trials and majority loss; 126 overview and 298 node samples with no capacity violation, maximum 85 A) and then failed at the PostgreSQL phase, because a psql row count that the script gives 15 s took over 60 s and the Docker engine then crashed and restarted; the script's own restore step also failed, so the stack was restored by hand. The fair-sharing phase and the load phases therefore have no Docker figures for the fair-sharing build. The cause is the development laptop, which had well under 1 GB of free RAM with some forty containers from several projects running; the engine crashed four times during this work. The Docker engine's problem is a property of that machine, not an established property of the system. As a substitute, scripts/native_cluster_check.py starts three real coordinator processes on localhost (real Raft over TCP and the real HTTP API, no database, no Docker) and plays the gateways against them; artifacts/native-cluster-check.json records one passing run of ten checks. The cluster committed fair_share on its own once every peer reported support; the policy endpoint answered 403 to a missing and to a wrong admin token; under first_come one building held 102 A while the others held 7 A and 4 A; after switching to fair_share the three buildings settled at 40/40/40 A in 13.1 s (checked against an independent max-min calculation, with the recorded wants required to match the demand sent); a changed demand mix of 10/100/100 A settled at 10/55/55 A in 12.5 s; after the leader was killed a new leader was elected in 3.1 s, the gateways were granted 6 further requests and the shares held; the restarted node rejoined and all three replicas ended with the same leases, wants, policy and applied index; and in 211 samples of the three nodes the granted total never exceeded 120 A (maximum seen 120 A). This is one run on one machine with a shared clock and a loopback network, sampling about twice a second, and without MQTT, gateways, PostgreSQL, nginx or the control service; the Docker evaluation should be repeated on a host with more free memory.

The TypeScript/Vite production build passed. npm audit --omit=dev reported zero known vulnerabilities for the final dependency set (React 19, Radix Dialog and the self-hosted IBM Plex Sans and IBM Plex Mono fonts). This is a dependency-database result, not a comprehensive security certification.

An earlier dashboard design passed seventeen automated Chrome checks against the live stack; that was a different design and is not the evidence for the console below. The current console has a 40-check browser test (scripts/browser-check.mjs), an accessibility walkthrough (keyboard only, reduced motion), a contrast check and a single-file preview check; the last three passed against a local mock server and a simulated-data build. The 40-check test was run three times against the live Docker stack with fair sharing on. The first run failed at the cluster-details check (after 24 passes) and the second passed all 40 checks, including a real leader-failover scenario with every coordinator restored, zero uncaught JavaScript errors and live WebSocket frames; artifacts/browser-check.json is that run and artifacts/board-desktop.png, board-mobile.png and board-after-failover.png are its captures. The failure was not root-caused: I added a bounded wait for the drawer's facts, a plausible race with the next live frame, which is a guess. A third run, after that change, got past the cluster-details check but timed out in the leader-failover scenario after 32 passes while the laptop was out of memory (see the fair-sharing paragraph above). The two failed runs are in artifacts/browser-check-failed-runs.json. The test therefore passed in one of three live runs, and the other two failures coincided with an unstable host.

Dashboard: the Ishtirak Console

The dashboard is one React 19 screen drawn like an electrical single-line diagram, so there are no pages to click through. A top bar names the system, shows the live-feed state and holds the admission-policy switch (fair sharing or first come) and the About drawer. The Capacity panel is the visual form of the capacity proof: reserved amps against the 120 A limit as a gauge with a peak marker that stays at the highest reading seen, a violation counter, a one-line proof ("No violation observed ..."), the free, served and curtailed amps, and a fairness score. The diagram draws the generator, a bus line and one feeder per building with its breaker, a metering block (demand, reserved and fair share), the lease countdown and the lease tiers as a stepped bar that shortens as older tiers expire; the three coordinators are drawn as a Raft cluster with a vote strip that states the majority in words. A Coordinators table lists each node's role, term, applied index, last log index and reserved amps with an inline Stop or Start and Confirm. A dock holds one panel at a time: the Allocation log (audit events with search, a kind filter, export and a Full log drawer), the Failure tests (the four scenarios leader failover, majority loss, gateway crash and database outage, each behind an inline Confirm, and an Overload test that can run under either policy), the meter Simulation (10, 50 or 200 meters, a demand multiplier, and the database and broker switches), and a history Chart. Drawers give the cluster details, a building's details and the full log. Every scenario restores the services it stopped on completion, timeout, cancel or navigation. Every value is an observation from polling or the WebSocket and is labelled as such; stale data parks the gauge and shows "No fresh data", and no history is synthesized. Every status carries a symbol and a word, not colour alone, and there are light, dark and automatic themes.

A demo mode (?demo=1, or automatic when the backend cannot be reached) runs the same console on an in-browser simulator so the interface can be viewed without Docker. It carries a permanent "Simulated data" chip that the real mode never shows. npm run build:preview writes a single self-contained file (about 0.5 MB, no external requests) in demo mode. The design avoids the standard sidebar-and-cards dashboard on purpose, and an earlier skeuomorphic switchboard design was replaced because it read as a game rather than as professional operations software; the type is IBM Plex Sans and IBM Plex Mono, self-hosted. The stable data-testid hooks that the browser check relies on are listed in web/TESTIDS.md.

7. Requirement and rubric mapping

Brief requirement / criterion	Concrete implementation or evidence
Clear, significant problem	Concurrent building demand against one finite generator budget; Sections 1 and 3
At least three components (brief's example: database, application, logging service)	Meter/broker, gateways, three coordinators, PostgreSQL (database), replicated audit log projected to SQL (logging service), UI/controller; Section 2 and Compose
Data partitioning	MQTT topics and one gateway per building
Communication mechanisms	MQTT QoS 1, HTTP/JSON, WebSocket, Raft TCP, SQL
Concurrency and consistency	Ordered majority-committed FSM, idempotency, atomic capacity checks; randomized fault check of the recorded history
Fault tolerance	Leader replacement, majority-loss refusal, safe lease expiry, SQL catch-up; repeated failover and reclamation trials; randomized fault schedule with a majority-loss window
Performance and scalability evaluation	10/50/200 meter results over five trials with mean, sd and range; concurrency sweep to 64 clients; end-to-end telemetry latency; explicit methodological limits
Challenges and trade-offs	Unknown outcomes, duplicates, stale leaders, monotonic timing, safety versus availability, fair-share allocation with tiered, decaying reservations, and retention; Sections 3-6
Creativity and practical relevance	Neighborhood-specific capacity coordination with a live operator, guided failure tests that break the running system on purpose, a max-min fair-sharing policy committed through Raft, and a capacity proof drawn from observations
Functional application	docker compose up --build -d; local dashboard at port 8088
Documentation	This report, README, source tests, measured JSON evidence
PowerPoint presentation	docs/Ishtirak_Presentation.pptx, with architecture, why it is distributed, safety, the live console and failure tests, results, trade-offs and limits, and speaker notes

The project addresses each required deliverable and the named distributed-systems concepts. This mapping is evidence of coverage, not a promise of a particular rubric score.

8. Reproduction and demonstration

Start Docker Desktop with Linux containers. In the project root run docker compose up --build -d, then open http://localhost:8088. Internet access is needed for the first image/dependency build. No host Go compiler or external AI credential is required.

Run python scripts/evaluate.py to repeat the fault and load sequence (about 7 to 12 minutes; the recorded run took 7 minutes, and a failing run is bounded by phase deadlines). Run python scripts/chaos_check.py --seed 1 --duration 90 for the randomized fault check; the same seed reproduces

the same fault schedule, and --dry-run prints it without touching the stack. Run them on a demo you are prepared to interrupt: they temporarily stop only this project's named services, preserve volumes, and restore services and the prior meter configuration. Do not change meter counts in the UI while they are measuring. Reservations decay on their own about 12 s after demand falls; python scripts/reset_demo.py restarts the gateways for an immediate clean state (see Section 4). Python unit tests: python -I -m unittest discover -s scripts -p "test_*.py"; dashboard tests: npm test in web.

Run node scripts/browser-check.mjs for browser checks. This uses Node 24 and local Chrome; set CHROME_PATH if Chrome is installed elsewhere. It creates a dedicated temporary browser profile under artifacts, changes the simulator to 10 and back to 50, and closes the test browser. It does not use the user's normal browser profile.

For a live presentation, open the Failure tests tab and run the scenarios in this order, letting each recover before the next: leader failover (the observed timeline shows the leader lost and a new one observed while the capacity proof stays at zero violations), majority loss (new grants stop and service reaches 0 A), gateway crash (conservative reclamation), then database outage (catch-up). Then compare 10/50/200 meters in the Simulation tab, run the Overload test under first come and then under fair sharing (the buildings converge on equal shares), and watch a building's reserved amount step down after demand falls (the Allocation log shows "Reduced" events). The dashboard distinguishes requested demand, served demand, and reserved capacity.

Use docker compose stop and docker compose start to preserve the demo between sessions. Do not remove volumes unless intentionally resetting all prototype data. The project does not publish or deploy to an external service.

9. Engineering review and remaining limits

Implementation used parallel workers in isolated copies for coordinator and telemetry changes; the main author integrated their bounded paths. An independent reviewer challenged the lease design and checked safety corrections, including replicated capacity initialization and a fixed lease protocol. Deterministic tests cover these changes.

The later additions (the redesigned console, its failure tests, the randomized fault checker and the repeated-trial evaluation) did not modify consensus, ledger or lease code. They were built by isolated workers, integrated by the main author and checked by running the project's own tests and the live runs described in Section 6; running them exposed and fixed three defects in the new evaluation code itself (a probe that reused a still-leased building, a timing signal capped by the lease budget, and a trial that assumed a fixed building held the lease). An independent read-only audit then compared the deliverables with the brief, its rubric and every figure in this report; its confirmed findings were fixed. No external review of the safety design has been performed, and the independent reviews do not amount to a formal verification.

Known design limitations remain: no physical electrical certification; no host, broker, database or gateway failover; potentially lost readings during outages; histories grow over time; fair sharing has no priority classes and building ids are not authenticated, so a client that can reach a coordinator can invent buildings (bounded; see Section 4); reservations decay only about one 12 s hold after demand falls, so a starved building waits about that long for its share; coordinators must be upgraded together for the tier and fair-share commands; a measured decision-throughput ceiling of about 47 per second whose bottleneck was not profiled; observed UI state may lag; and no production authentication/TLS design.

Optional image/TypeSafe ideas were not added to authoritative allocation or to the prototype's scope.

The next engineering steps, if scope is expanded, are repeated multi-host testing, clock/suspension experiments, bounded history retention with replay guarantees, priority classes for fair sharing, authenticated gateways, and a separate authenticated deployment design. These are future work, not implemented capabilities.

10. Sources and evidence index

• Final-Project 2025.pdf: course brief, required deliverables, evaluation rubric.

• Ishtirak_Project_Proposal_slim.docx: proposed architecture, invariant, experiments, limited scope.

• Ishtirak_Project_Flow.drawio: original project flow.

• HashiCorp Raft API: https://pkg.go.dev/github.com/hashicorp/raft

• Eclipse Paho Go MQTT API: https://pkg.go.dev/github.com/eclipse/paho.mqtt.golang

• internal/ledger/ledger.go and its tests: authoritative transitions and replay.

• internal/coordinator/server.go, deadlines.go and tests: Raft lifecycle, grants and expiry.

• internal/telemetry/gateway.go and tests: safe local use, retries and buffering.

• scripts/evaluate.py, artifacts/evaluation.json: measured fault/load results, repeated trials, concurrency sweep, end-to-end latency and raw response timings.

• scripts/chaos_check.py, artifacts/chaos-check.json: seeded randomized fault schedule, recorded history and invariant check.

• scripts/reset_demo.py: returns the demo to a fair allocation after a demand spike.

• web/src/board/, web/src/lab/scenarios.ts: the console, guided failure scenarios, capacity proof and observed timeline.

• docs/FAIR_SHARING.md: fair-sharing rules, safety argument, review findings and limits.

• scripts/native_cluster_check.py, artifacts/native-cluster-check.json: three real coordinator processes (no Docker): policy commit, token refusal, hog to 40/40/40 and 10/55/55, leader kill and rejoin, replica agreement, capacity sampling.

• scripts/browser-check.mjs, artifacts/browser-check.json, artifacts/board-*.png: the console's 40 browser assertions on the live stack (one passing run) and its captures; artifacts/browser-check-failed-runs.json: the two live runs that did not pass.

• artifacts/evaluation-fair-share-partial.json: the live evaluation of the fair-sharing build, which stopped after four passing phases (see Section 6).

• README.md, compose.yaml: startup, operation, configuration and limits.